

Báo cáo bài tập D2

Nội dung báo cáo Project – D2 – Nộp Tài liệu Yêu cầu & Bài toán

Lĩnh vực của nhóm chọn: Hệ thống đặt vé sự kiện / Rạp chiếu phim

1. Xác nhận lĩnh vực & Bối cảnh

Lĩnh vực: Hệ thống đặt vé sự kiện / địa điểm – cụ thể là hệ thống quản lý và đặt vé cho chuỗi rạp chiếu phim đa cụm rạp.
Hệ thống phục vụ hai nhóm đối tượng chính:

- Khách hàng: tra cứu phim, lịch chiếu, đặt vé, hoàn vé combo, xem lại lịch sử các phim xem gần đây.
- Nhân viên tại rạp: kiểm tra mã vé, xác nhận trạng thái vé. Do hệ thống phục vụ nhiều cụm rạp trên toàn quốc, dữ liệu có thể lên tới hàng triệu vé, suất chiếu và lịch sử giao dịch. Vì vậy, hệ thống cần sử dụng các cấu trúc dữ liệu phù hợp để đảm bảo tốc độ truy xuất và xử lý hiệu quả.

2. Các yêu cầu bắt buộc chung (MC1 & MC2)

MC1: Tra cứu chính xác ở quy mô lớn – Kiểm tra vé tại cổng

Bối cảnh:

- Khi khách hàng đến rạp, nhân viên soát vé quét Booking Reference ID trên vé, bao gồm các thông tin định danh cho vé (ví dụ: VN-CINEMA-98234A) bằng máy quét, việc soát vé phải được tra cứu ngay lập tức trong cơ sở dữ liệu của hệ thống vé, để kiểm tra tính hợp lệ và trả về thông tin tương ứng để nhân viên quyết định cho phép khách hàng vào phòng chiếu hay từ chối.
- Một mô hình dữ liệu vé bao gồm:

Trường	Giải thích	Kiểu dữ liệu	Ví dụ
booking_id	Mã vé	String (VN-CINEMA-XXXXXY)	VN-CINEMA-12345A
customer_name	Tên khách hàng	String	Nguyen_Van_Muoi
movie_name	Tên bộ phim	String	MINIONS
showtime	Thời gian bắt đầu chiếu	String (YYYY-MM-DD HH:MM)	2026-09-03 21:36
cinema_address	Địa điểm rạp chiếu phim	String	THUDUC
cinema_room	Phòng chiếu phim	String	ROOM02
seats	Ghế ngồi đã đặt	String	A01
ticket_status	Trạng thái của vé	String (VALID / USED)	USED

Giả định: Mỗi booking_id hợp lệ về định dạng đã được nạp vào hệ thống là duy nhất, tức là không có 2 bản ghi trùng booking_id trong dữ liệu gốc
Quy tắc kiểm tra vé: Vì một vé có thể vi phạm nhiều điều kiện cùng lúc, nên ta phải kiểm tra theo đúng thứ tự quy ước sau, dừng lại và trả về lập tức khi gặp điều kiện đầu tiên bị vi phạm:

- Kiểm tra định dạng mã vé có hợp lệ
- Kiểm tra mã vé có tồn tại trong hệ thống
- Kiểm tra mã vé đã hết hay chưa (Suất chiếu đã kết thúc so với thời điểm đã quét hay chưa)
- Kiểm tra mã vé đã được dùng hay chưa
- Nếu vượt qua tất cả các bước trên, tức mã vé có hiệu lực (VALID) Quy tắc xác định thời điểm suất chiếu đã kết thúc: một vé được coi là hết hạn (EXPIRED) nếu thời điểm quét vé vượt quá showtime + 3 giờ (quy ước mọi suất chiếu có thời lượng cố định 3 giờ).

Input:

Dòng 1: Số nguyên N — số lượng vé đã phát hành trong hệ thống. N dòng tiếp theo: mỗi dòng là một bản ghi vé, các thông tin ghi tách nhau bởi dấu cách (" ")

booking_id|customer_name|movie_name|showtime|cinema_address|cinema_room|seats|ticket_status Dòng tiếp theo: Số nguyên Q — số lượt vé cần soát vé tại cổng

Q dòng tiếp theo: mỗi dòng là một chuỗi Booking Reference ID cần kiểm tra Dòng cuối cùng: CURRENT_TIME (Thời điểm quét vé) — mốc thời gian hiện tại tại thời điểm quét (định dạng YYYY-MM-DD HH:MM)

Output:

Với mỗi lượt soát vé, nếu mã vé hợp lệ in ra đúng một dòng kết quả: VALID Nếu mã vé không hợp lệ, hệ thống trả về một trong các mã lỗi sau:

Mã lỗi	Điều kiện vi phạm
INVALID_FORMAT	Booking_id không khớp định dạng quy định
NOT_FOUND	Booking_id đúng định dạng nhưng không tìm thấy trong hệ thống
EXPIRED	Suất chiếu (showtime + 3h) đã kết thúc so với thời điểm quét
USED	Vé đã được sử dụng trước đó (ticket_status = USED)

Ràng buộc:

- $1 \leq N \leq 10^6$ (tổng số vé trong hệ thống)
 - $1 \leq Q \leq 10^5$ (số lượt quét vé tại cổng)
 - Độ dài customer_name, movie_name, cinema_address từ 1 đến 50 ký tự (không được chứa khoảng trống)
 - booking_id có định dạng kiểu VN-CINEMA-XXXXXY trong đó XXXXX là đúng 5 chữ số (0-9) và Y là đúng 1 chữ cái in hoa (A-Z)
 - showtime có định dạng (YYYY-MM-DD HH:MM)
 - Việc tra cứu không được phép làm thay đổi bất kì dữ liệu nào của vé.
-

Trường hợp ngoại lệ (Edge cases):

- booking_id sai định dạng (thiếu ký tự, sai độ dài, chữ thường, thiếu tiền tố VN-CINEMA-)
- booking_id đúng định dạng nhưng không tồn tại trong dữ liệu N vé
- Vé đã hết hạn vì suất chiếu đã kết thúc (EXPIRED)

- Vé đã được sử dụng trước đó (USED)
- Lỗi định dạng (showtime, cineme_room, seats, current_time) không đúng với quy ước.
- Một booking_id được quét nhiều lần trong cùng Q lượt quét.
- Showtime rơi vào ngày 29/2 của năm không nhuận, hoặc định dạng hợp lý về chuỗi nhưng vô lý về mặt thực tế (Ví dụ: 2026-13-32 25:67)
- Danh sách ghế seats bị lặp lại (Ví dụ: A01,A02,A01)
- Check-in vé quá sớm, tức là CURRENT_TIME nhỏ hơn showtime quá nhiều ngày, thì hệ thống cho phép hay báo lỗi quá sớm

MC2: Truy xuất theo thứ tự – Hàng đợi ưu tiên xử lý yêu cầu đặt ghế đồng thời

Bối cảnh

Khi các bộ phim bom tấn mở bán vé, hàng loạt khách hàng đồng loạt truy cập chức năng đặt vé online, dẫn đến lượng lớn dữ liệu cho hệ thống các rạp phim. Không thể tránh khỏi việc cùng một thời điểm và cùng một vị trí ghế ngồi có hơn một khách hàng đặt vé thành công cho cùng một bộ phim.

Để giải quyết vấn đề bán trùng vé và đảm bảo sự nhất quán dữ liệu, hệ thống không xử lý đồng thời các thao tác trên cùng một tài nguyên mà phải đưa toàn bộ yêu cầu vào hàng đợi ưu tiên để xử lý tuần tự theo đúng trình tự tiếp nhận.

Mô hình dữ liệu của một yêu cầu đặt ghế bao gồm:

Trường	Giải thích	Kiểu dữ liệu	Ví dụ
request_id	Mã định danh duy nhất của yêu cầu	String (REQ-XXXXX)	REQ-00001
showtime_id	Mã suất chiếu	String	SHOWTIME-482
seat_id	Mã ghế khách hàng muốn đặt	String	A01
customer_id	Mã khách hàng gửi yêu cầu	String	CUS-10293
timestamp	Thời điểm gửi yêu cầu	String (YYYY-MM-DD HH:MM:SS)	2026-09-03 21:36:05
request_status	Trạng thái hiện tại của yêu cầu	String (PENDING / CANCELLED)	PENDING

Giải định:

- Mỗi request_id là duy nhất trong toàn bộ dữ liệu đầu vào.
- Mỗi seat_id chỉ có ý nghĩa trong phạm vi một showtime_id cụ thể (hai suất chiếu khác nhau có thể trùng seat_id nhưng là hai ghế độc lập).

Quy tắc xử lý

Hệ thống lặp lại các bước sau cho đến khi hàng đợi không còn yêu cầu ở trạng thái PENDING (chờ xử lý). Tại mỗi bước, kiểm tra theo đúng thứ tự quy ước, dừng lại và trả về ngay kết quả tương ứng khi gặp điều kiện đầu tiên bị vi phạm:

1. Lấy yêu cầu ưu tiên nhất: Lấy ra yêu cầu đang ở trạng thái **PENDING** có **timestamp** nhỏ nhất (sớm nhất) trong hàng đợi. Nếu có nhiều yêu cầu cùng **timestamp**, ưu tiên yêu cầu có **request_id** nhỏ hơn theo thứ tự từ điển.
2. Kiểm tra định dạng (**INVALID**): Kiểm tra **request_id** và **seat_id** có đúng định dạng quy định hay không.
3. Kiểm tra trạng thái hủy (**CANCELLED**): Kiểm tra yêu cầu có đang ở trạng thái **CANCELLED** tại thời điểm được lấy ra hay không.
4. Kiểm tra trạng thái ghế (**SEAT_TAKEN**): Kiểm tra ghế (**seat_id** trong **showtime_id** tương ứng) đã được một yêu cầu khác khóa thành công trước đó hay chưa.
5. Khóa ghế thành công (**LOCKED**): Nếu vượt qua tất cả các bước trên, khóa ghế cho yêu cầu hiện tại và đánh dấu ghế đó là đã khóa.

Input

- Dòng 1: Số nguyên M — số lượng yêu cầu đặt giữ chỗ gửi đến hệ thống.
- M dòng tiếp theo: Mỗi dòng là một yêu cầu, các thông tin ghi phân tách nhau bởi dấu gạch đứng (**|**):
request_id|showtime_id|seat_id|customer_id|timestamp|request_status

Ràng buộc thứ tự đầu vào: Các dòng yêu cầu trong input không nhất thiết được liệt kê theo thứ tự **timestamp** tăng dần — hệ thống phải tự sắp xếp lại theo đúng **timestamp** và **request_id** trước khi xử lý.

Output

Với mỗi yêu cầu, xử lý lần lượt theo đúng thứ tự **timestamp** tăng dần (nếu cùng thời điểm thì xét tới thứ tự của **request_id** theo từ điển), in ra đúng một dòng kết quả theo đúng thứ tự và ngăn cách nhau bởi dấu **|**:
request_id|seat_id|customer_id|result_status

Trong đó **result_status** nhận một trong các giá trị sau:

Mã kết quả	Điều kiện vi phạm / Trạng thái
LOCKED	Ghế được khóa thành công cho yêu cầu này.
SEAT_TAKEN	Ghế đã được khóa bởi một yêu cầu khác có timestamp sớm hơn (hoặc cùng timestamp nhưng thắng tie-breaker theo request_id).
CANCELLED	Yêu cầu đã bị hủy trước khi được xử lý tới lượt.
INVALID	request_id hoặc seat_id không đúng định dạng quy định.

Ràng buộc

- $1 \leq M \leq 10^5$ (số lượng yêu cầu đặt ghế trong một đợt mở bán).
- **request_id** có định dạng kiểu **REQ-XXXXX** trong đó **XXXXX** là đúng 5 chữ số (**0-9**).
- **seat_id** có định dạng kiểu **[A-L][0-9]{1,2}** (Ví dụ: **A01**, **L12**).
- **timestamp** có định dạng **YYYY-MM-DD HH:MM:SS**, đảm bảo có thể so sánh trực tiếp theo thứ tự từ điển tương ứng đúng với thứ tự thời gian thực tế.
- Hệ thống phải đảm bảo một **seat_id** trong một **showtime_id** chỉ được khóa thành công cho duy nhất một **request_id**.

Trường hợp ngoại lệ (Edge cases)

- Yêu cầu bị hủy (**CANCELLED**) trước khi đến lượt được xử lý.
- Nhiều yêu cầu cùng nhắm đến một **seat_id** trong cùng một **showtime_id**.
- Một **customer_id** gửi nhiều yêu cầu cho cùng một **seat_id**.
- Các dòng yêu cầu trong dữ liệu đầu vào không được sắp xếp sẵn theo **timestamp** — yêu cầu có **timestamp** sớm hơn nhưng xuất hiện sau trong input vẫn phải được xử lý trước.
- **request_id** hoặc **seat_id** sai định dạng quy định.
- Người dùng dùng tool để một **customer_id** tạo ra nhiều requests cho cùng 1 **seat_id** với **timestamp** giống hệt nhau (xử lý ưu tiên theo **request_id** từ điển).
- Ghế (**seat_id**) không tồn tại trong sơ đồ phòng.
- Hai dòng input giống hệt nhau 100% từ **request_id** đến **timestamp** do lỗi nhân đôi yêu cầu.
- **timestamp** rơi vào ngày 29/02 của năm không nhuận, hoặc định dạng hợp lý về chuỗi nhưng vô lý về mặt thực tế (Ví dụ: **2026-13-32 25:62:61**).
- Một người đặt nhiều ghế cùng một lúc.

2. Các yêu cầu do nhóm tự phát hiện (Self-discovered Requirements)

YC1: Tính năng “Hoàn tác” (Undo) thao tác chọn ghế / combo

Bối cảnh:

- Khi khách hàng đặt vé xem phim trên ứng dụng điện thoại, khách hàng có thể thực hiện nhiều thao tác như chọn ghế, bỏ chọn ghế, thêm combo bắp nước hoặc thay đổi số lượng combo. Do sơ đồ phòng chiếu thì có nhiều chi tiết mà xem qua màn hình điện thoại thì nhỏ, khách hàng có thể vô tình chọn nhầm ghế hoặc thay đổi số lượng combo không mong muốn. Vì vậy, hệ thống cần cung cấp chức năng “Hoàn tác” để cho phép khách hàng hoàn tác thao tác gần nhất, đưa trạng thái giỏ hàng hay sơ đồ ghế về đúng trạng thái ngay trước khi thao tác đó được thực hiện. Giả định chức năng hoàn tác có thể lưu trữ tối đa 10 bước và chỉ có thể hoàn tác thao tác đã bấm trong 15 phút.

Giả sử tình huống: Khách hàng chọn ghế A01 rồi chọn tiếp ghế A02 thêm một combo bắp nước.

- Ví dụ: Ghế A01 ☐ Ghế A02 ☐ Combo bắp nước.

Trạng thái giỏ hàng

Combo bắp nước

Ghế A02

Ghế A01

- Hoàn tác lần 1: Ghế A01 ☐ Ghế A02. “Combo bắp nước đã bị xóa khỏi giỏ hàng” nên chỉ còn “Ghế A01” và “Ghế A02”.

Trạng thái giỏ hàng

Ghế A02

Ghế A01

- Hoàn tác lần 2: “Ghế A02” đã được hoàn tác và chỉ còn “Ghế A01”.

Trạng thái giỏ hàng

Ghế A01

Input:

- Dòng 1: số nguyên N – số lượng combo và ghế đã đặt trong hệ thống.
- Dòng 2: chuỗi CURRENT_TIME – thời điểm thực hiện thao tác (định dạng YYYY-MM-DD HH:MM).
- N dòng tiếp theo: mỗi dòng là một bản ghi, các thông tin được ghi cách nhau bởi "|", theo dạng `operation|order_time`.
- Dòng tiếp theo: số nguyên Q – số lượng hoàn tác. Các dòng sau đó là lệnh **UNDO**.

Output:

Với mỗi lần bấm hoàn tác, nếu thao tác thành công thì hệ thống sẽ in ra “SUCCESSFULL OPERATION”. Còn nếu thao tác không hợp lệ thì hệ thống sẽ in ra “FAILED DATA”. Khi xong tất cả thao tác **UNDO** sẽ in ra lại các thao tác còn trong stack

Ràng buộc:

- CURRENT_TIME (thời gian thực hiện hoàn tác) phải lớn hơn hoặc bằng về mặt thời gian so với các order_time
- Thao tác được thực hiện cuối cùng phải được hoàn tác đầu tiên
- Thêm một thao tác mới vào lịch sử phải là thao tác đơn (tức là không thực hiện hai thao tác cùng một thời điểm).
- Lịch sử hoàn tác chỉ lưu tối đa 10 thao tác gần nhất để giới hạn bộ nhớ sử dụng.
- Khi số lượng vượt qua 10 thao tác thì thao tác cũ nhất sẽ bị loại khỏi lịch sử.
- Lịch sử thao tác chỉ lưu ở bộ nhớ tạm nên việc khiến ứng dụng khởi động lại sẽ mất hết dữ liệu trong danh sách hoàn tác (tức không thể hoàn tác).
- Trạng thái sau khi hoàn tác phải đúng chính xác với trạng thái ngay trước thao tác được hoàn tác.

Trường hợp ngoại lệ:

- Người dùng nhấn “hoàn tác” khi chưa có thao tác nào trong lịch sử thì hệ thống in ra “FAILED DATA”.
- Thao tác cần hoàn tác đã tồn tại quá 15 phút kể từ thời điểm thực hiện (so với CURRENT_TIME) thì thao tác đó không còn hợp lệ để hoàn tác, hệ thống in ra “FAILED DATA”.

- Số lần yêu cầu hoàn tác nhiều hơn số thao tác còn có thể hoàn tác trong lịch sử (do lịch sử đã trống hoặc các thao tác còn lại đều đã hết hạn 15 phút) thì các lần hoàn tác dư đó hệ thống in ra “FAILED DATA”.
- Thao tác muốn hoàn tác thuộc về một đơn hàng đã được xác nhận hoặc thanh toán thành công thì hệ thống từ chối thực hiện hoàn tác và in ra “FAILED DATA”.
- Dữ liệu đầu vào không hợp lệ (ví dụ: CURRENT_TIME hoặc order_time sai định dạng YYYY-MM-DD HH:MM, N hoặc Q âm, chuỗi operation rỗng, ...) thì hệ thống in ra “FAILED DATA”.
- CURRENT_TIME nhỏ hơn hoặc bằng order_time của thao tác gần nhất trong lịch sử (thời gian hoàn tác không hợp lệ so với thời điểm thao tác được ghi nhận) thì hệ thống in ra “FAILED DATA”.
- Khách đã chọn ghế rồi bỏ chọn ghế, sau đó hoàn tác lại để chọn lại ghế đó. Trong lúc hoàn tác thì khách hàng khác đã đặt ghế đó.
- CURRENT_TIME rơi vào ngày 29/2 của năm không nhuận, hoặc định dạng hợp lý về chuỗi nhưng vô lý về mặt thực tế (Ví dụ: 2026-13-32 25:66)

YC2: Danh các phim vừa xem gần đây (Recently Viewed Movies).

Bối cảnh:

- Để tăng trải nghiệm cá nhân hóa cho khách hàng, hệ thống cần lưu lại lịch sử các bộ phim mà người dùng vừa xem thông tin trên ứng dụng. Khi khách hàng mua vé của một bộ phim, Movie ID của bộ phim đó được thêm vào danh sách "Các phim vừa xem gần đây".
- Danh sách chỉ lưu tối đa 5 bộ phim gần nhất, được sắp xếp theo thứ tự từ phim vừa xem gần nhất đến phim xem lâu nhất.
- Nếu khách hàng mua lại một bộ phim đã có trong danh sách, hệ thống không tạo thêm bản sao mà đưa bộ phim đó lên vị trí đầu tiên. Nếu danh sách đã có đủ 5 phim và khách hàng xem một phim mới, phim ở vị trí cuối cùng sẽ bị loại bỏ.
- Do đây là bộ nhớ tạm có kích thước nhỏ và thường xuyên được cập nhật, hệ thống cần sử dụng cấu trúc dữ liệu phù hợp để thao tác thêm, xóa và cập nhật vị trí nhanh chóng

Input:

- Dòng 1: số nguyên N, M – Số ID tồn tại trong danh sách, Số ID phim mới xem.
- N dòng tiếp theo chứa ID các phim coi từ mới nhất nhất đến cũ nhất.
- M dòng tiếp theo chứa ID của các mới xem.

Output

- Nếu lịch sử xem của khách hàng có tồn tại ít nhất 1 bộ phim thì
- In ra toàn bộ phim có trong danh sách theo trình tự thời gian, mỗi hàng một ID phim.
- Còn nếu lịch sử xem phim trống thì in: “Không có lịch sử xem phim gần đây”

Ràng buộc

- Danh sách tối đa 5 phần tử. $0 \leq N \leq 5$
- Danh sách không chứa phần tử trùng lặp.
- Quy ước vị trí 0 là phim xem gần nhất

- Phim được coi từ danh sách được đẩy lên vị trí đầu tiên không thêm mới.
- Với kích thước danh sách tối đa 5 phần tử, thao tác phải nhanh và không làm ảnh hưởng dữ liệu tổng thể.

Trường hợp ngoại lệ (Edge casse)

- Tài khoản mới tạo chưa tồn tại danh sách đã xem.
- Tài khoản mua vé nhưng không check-in xem phim.

YC3: Lọc suất chiếu theo khung giờ

Lý do phát hiện

Một bộ phim có thể có nhiều suất chiếu tại nhiều phòng và cụm rạp khác nhau. Khách hàng thường chỉ có thể xem phim trong một khoảng thời gian nhất định. Ví dụ, khách muốn xem "Dune: Part Two" vào tối thứ Bảy từ 18:00–23:00. Nếu hệ thống trả về toàn bộ suất chiếu, khách phải tự lọc các suất không phù hợp. Vì vậy, hệ thống cần hỗ trợ lọc suất chiếu theo khung giờ, giúp người dùng nhanh chóng tìm được suất phù hợp. Đồng thời, khi số lượng suất chiếu lớn, hệ thống cần tìm kiếm theo khoảng thời gian hiệu quả thay vì duyệt toàn bộ dữ liệu.

Bối cảnh

Khách hàng muốn xem phim "Dune: Part Two" vào tối thứ Bảy và yêu cầu hiển thị các suất chiếu từ 18:00 đến 23:00, sắp xếp theo thời gian tăng dần.

Input

Dòng 1: Số nguyên N — số lượng suất chiếu. N dòng tiếp theo:

MovieID|Date|ShowtimeID|CinemaName|CinemaRoom|StartTime|EndTime Dòng tiếp theo: Truy vấn nhập từ bàn phím, lần lượt từng giá trị: **MovieID**, **Date**, **T1**, **T2** (nhập **exit** ở MovieID để thoát).

Ví dụ: MovieID = DUNE2 Date = 2026-09-05 T1 = 18:00 T2 = 23:00

Output

- Với mỗi truy vấn, in ra các Showtime thỏa mãn: • MovieID = MovieID yêu cầu • Date = Date yêu cầu • $T1 \leq \text{StartTime} \leq T2$
- Định dạng: ShowtimeID|CinemaName|CinemaRoom|StartTime|EndTime
- Các kết quả được sắp xếp theo StartTime tăng dần, nếu nhiều suất có cùng StartTime thì sắp xếp tiếp theo ShowtimeID rồi đến CinemaRoom.
- Nếu không có suất phù hợp (hoặc dữ liệu truy vấn không hợp lệ) thì trả về danh sách rỗng và in "NOT FOUND!!".

Ràng buộc

- $1 \leq N \leq 10^6$.
- MovieID, ShowtimeID, CinemaName, CinemaRoom không rỗng và không chứa ký tự '|'.

- MovieID không chứa khoảng trắng và có độ dài từ 1 đến 50 ký tự.
 - Date có kiểu định dạng YYYY-MM-DD
 - StartTime và EndTime có định dạng HH:MM.
 - T1 và T2 có định dạng HH:MM, thuộc cùng một ngày quy ước tính theo 24 giờ
 - $T1 \leq T2$
 - Khoảng thời gian [T1, T2] bao gồm hai đầu mút và áp dụng cho StartTime.
 - Thao tác tìm kiếm không được làm thay đổi dữ liệu gốc.
-

Trường hợp ngoại lệ (Edge cases)

- Trong trường hợp $T1 = T2$, chỉ lấy các suất có StartTime đúng bằng T1.
 - $T1 > T2$: trả về rỗng.
 - MovieID không tồn tại: trả về rỗng.
 - Ngày yêu cầu không có lịch chiếu: trả về rỗng.
 - Thời điểm StartTime của phim trùng khớp chính xác từng phút với T1 hoặc T2
 - Cùng một thời điểm một bộ phim chiếu ở nhiều phòng: sắp xếp theo ShowtimeID, rồi đến CinemaRoom.
 - Date rơi vào ngày 29/2 của năm không nhuận, hoặc định dạng hợp lệ về chuỗi nhưng vô lí về mặt thực tế (Ví dụ: 2026-13-32): trả về rỗng.
 - Giờ có định dạng đúng nhưng vô lí (Ví dụ: 24:00, 12:60): trả về rỗng.
-